

Optimizing Patch Selection in HMAX Using Genetic Algorithms

Abstract

The human visual system is remarkably efficient in recognizing objects under a wide range of conditions. Inspired by its hierarchical architecture, computational models like HMAX attempt to replicate this layered processing by extracting increasingly complex features at successive stages. A crucial component of such models is the selection of image patches used to form intermediate representations. Traditionally, these patches are selected randomly, which can result in redundant or non-informative features. In this work, we implement the HMAX model and explore an evolutionary approach to enhance its performance. Specifically, we use a genetic algorithm (GA) to identify a compact and informative subset of patches that are more effective for object recognition. Our results across multiple object recognition tasks show that GA-based patch selection improves classification accuracy while reducing the dimensionality of the feature space.

1 Introduction

Understanding how the brain recognizes objects has long been a major challenge in both neuroscience and artificial vision. Despite their complexity, the human and primate visual systems can recognize objects quickly and robustly, even in cluttered or variable environments. This ability has inspired many computational models, especially those that attempt to mimic the brain’s hierarchical structure.

One of the well-known biologically inspired models is HMAX, which simulates the ventral visual pathway by stacking layers of increasingly complex processing units. HMAX extracts low-level features (like edges and bars) in early stages and builds up to more abstract representations through intermediate layers using simple and complex cells. A critical part of this pipeline is the use of image patches extracted from C1 responses, which serve as the basis for later feature representation and classification.

In standard implementations, these patches are often chosen randomly from training images. While this approach is easy to implement, it may result in redundant or irrelevant features, which can limit the model’s performance. Prior studies have explored various strategies to improve patch selection, such as clustering or linear combinations, but many of these approaches either lack biological plausibility or are task-specific.

In this work, we take a different approach. We incorporate a Genetic Algorithm (GA) into the HMAX model to automatically select a subset of patches that are more informative for object recognition. GAs are inspired by natural evolution and have shown success in optimizing complex systems. By using classification accuracy as a fitness function, we aim to evolve a patch set that improves recognition performance while keeping the feature space compact. Our method is general, task-independent, and biologically motivated.

2 Materials and Methods

2.1 Overview of the HMAX Model

The biologically inspired HMAX model is a hierarchical framework designed to simulate the ventral visual pathway in the primate brain. Originally proposed based on the work of Hubel and Wiesel, it emulates the progression from the primary visual cortex (V1) to higher-order areas such as the inferior temporal (IT) cortex and prefrontal cortex (PFC). The model architecture alternates between layers of simple (S) and complex (C) units, forming four main stages: S1, C1, S2, and C2.

S units enhance feature selectivity by combining inputs through Gaussian-like operations, while C units introduce position and scale invariance using a max pooling strategy. As information flows through the layers, both selectivity and invariance increase, mimicking the biological transformation from edge detection in V1 to object-level recognition in IT.

In the following sections, we describe the implementation details of each layer, including the specific functions, parameters, and spatial configurations used in our version of the HMAX model.

2.2 S1 Layer

The S1 layer of the HMAX model simulates the behavior of simple cells in the primary visual cortex (V1). These cells respond to oriented bars or edges within their receptive fields. In computational terms, S1 units apply a bank of Gabor filters to the input image to detect low-level features such as orientation, frequency, and position.

Each Gabor filter is defined by the following function:

$$F(x, y) = \exp\left(-\frac{x_0^2 + \gamma^2 y_0^2}{2\sigma^2}\right) \cos\left(\frac{2\pi x_0}{\lambda}\right)$$

where:

$$x_0 = x \cos \theta + y \sin \theta, \quad y_0 = -x \sin \theta + y \cos \theta$$

The parameters of the Gabor function control the properties of the filter:

- θ : the preferred orientation of the filter
- γ : the aspect ratio (set to 0.3)
- σ : the standard deviation of the Gaussian envelope (controls bandwidth)
- λ : the wavelength of the cosine component (controls frequency)

In our implementation, we used Gabor filters at four orientations: $0^\circ, 45^\circ, 90^\circ, 135^\circ$, consistent with neurophysiological findings. The filters are applied across eight different scales, resulting in a total of 64 different S1 maps ($8 \text{ scales} \times 4 \text{ orientations}$).

Each S1 map corresponds to a filtered version of the input image at a particular scale and orientation. The scale bands range from small (7×7) to large (37×37) receptive fields, covering local to global spatial frequency content. As shown in Table 1, each scale band is associated with specific Gabor parameters σ and λ to model the tuning properties of V1 simple cells. The filter parameters were chosen based on prior physiological data to ensure biological plausibility.

The output of the S1 layer is thus a set of 64 feature maps, each emphasizing edges or bars at specific scales and orientations. These feature maps serve as the input to the subsequent C1 layer, where local pooling is applied to build invariance to small translations.

2.3 S1 Layer: Implementation Details

To simulate the behavior of orientation-selective simple cells in the primary visual cortex (V1), we constructed a full bank of Gabor filters representing different scales and orientations. Our implementation follows the biological principles underlying the HMAX model and

uses hand-crafted parameters grounded in neurophysiological evidence.

We defined eight scale bands, each consisting of two distinct filter sizes and associated Gabor parameters. For every filter size within a band, we applied Gabor filters at four canonical orientations: $0^\circ, 45^\circ, 90^\circ$, and 135° . This configuration yields a total of 64 Gabor filters ($8 \text{ bands} \times 2 \text{ sizes} \times 4 \text{ orientations}$), consistent with the hierarchical feature extraction mechanism in the ventral visual stream.

Each Gabor filter was defined by a fixed set of parameters: standard deviation σ , wavelength λ , orientation θ , and an aspect ratio $\gamma = 0.3$. These values were selected based on reported tuning properties of parafoveal simple cells in macaque V1. The filters were implemented as non-trainable 2D convolutional layers (Conv2d) with fixed weights initialized from the corresponding Gabor kernels. The spatial padding for each convolution matched half the kernel size to preserve spatial dimensions.

To improve computational efficiency, we implemented a caching mechanism. The output of the S1 layer for each input image was hashed using an MD5 fingerprint and stored to disk. Upon repeated evaluations of the same input, the cached result was directly retrieved, avoiding redundant computations.

During inference, the S1 layer applies all 64 filters to the input grayscale image and computes the absolute value of the convolution responses. This yields a set of 64 feature maps representing edge energy at multiple scales and orientations. These maps form the basis for further processing in the C1 layer, where local pooling introduces spatial invariance.

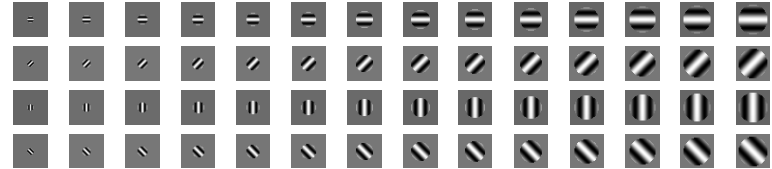


Figure 1: Structure of the S1 layer.

Table 1: Summary of the *S1* and *C1* SMFs Parameters

Scale Band	Pooling Grid ($N_S \times N_S$)	Overlap Δ_S	Filter Size	Gabor σ	Gabor λ
Band 1	8×8	4	$7 \times 7, 9 \times 9$	2.8, 3.6	3.5, 4.6
Band 2	10×10	5	$11 \times 11, 13 \times 13$	4.5, 5.4	5.6, 6.8
Band 3	12×12	6	$15 \times 15, 17 \times 17$	6.3, 7.3	7.9, 9.1
Band 4	14×14	7	$19 \times 19, 21 \times 21$	8.2, 9.2	10.3, 11.5
Band 5	16×16	8	$23 \times 23, 25 \times 25$	10.2, 11.3	12.7, 14.1
Band 6	18×18	9	$27 \times 27, 29 \times 29$	12.3, 13.4	15.4, 16.8
Band 7	20×20	10	$31 \times 31, 33 \times 33$	14.6, 15.8	18.2, 19.7
Band 8	22×22	11	$35 \times 35, 37 \times 37$	17.0, 18.2	21.2, 22.8

2.4 Visualizing S1 Responses

To qualitatively evaluate the behavior of the S1 layer, we applied the model to a sample grayscale image of an airplane. The original image, shown in Figure 2, is passed through the bank of Gabor filters defined in the S1 layer. Figure ?? displays a subset of the result-

ing feature maps corresponding to four different Gabor filters.

Original Image



Figure 2: Original Input Image used for testing the S1 layer.

2.5 C1 Layer

The C1 layer in the HMAX model corresponds to complex cells in the primary visual cortex, which are known for their larger receptive fields and tolerance to both position and scale. In our implementation, C1 units are designed to enhance this invariance by performing a two-stage max operation on the S1 outputs.

The first step involves pooling over local S1 responses that share the same orientation and scale band. Each scale band in the S1 layer consists of two filters with different sizes (e.g., 7×7 and 9×9 for Band 1), which generate corresponding S1 feature maps. These maps are then combined by calculating the pixel-wise maximum across them, resulting in 32 intermediate C1 maps, one for each of the four orientations and eight scale bands.

To further enhance translation invariance, a max-pooling operation is applied over non-overlapping or slightly overlapping spatial grids of size $N_S \times N_S$, where N_S increases from 8 (Band 1) to 22 (Band 8). The overlap between adjacent grid cells, denoted by Δ_S , also increases with scale. Within each grid, the maximum response among the pooled values is retained. This operation is designed to provide better scale and position invariance by reducing spatial resolution while preserving the most prominent features.

In our implementation, we handle the caching of C1 layer outputs to speed up repeated computations. Each output of the S1 layer is processed and stored in cache to avoid redundant calculations for identical inputs. This ensures that once a particular set of outputs has been computed, they are reused efficiently.

It is important to note that the dimensions of the pooled C1 maps vary based on the input image size and the pooling window size. For instance, with an input image of size 140×253 , the pooled output maps differ across bands. Band 0 yields pooled outputs of size 34×62 , while Band 7 produces 11×22 . This variation reflects the non-uniform receptive field sizes found in biological systems, which is captured in the model’s hierarchical structure.

Finally, the 32 C1 maps generated by this two-stage max operation are passed forward to the S2 layer, where they are compared with learned prototypes for object recognition tasks.

Table 2: Pooled C1 Output Sizes by Band (Input Image: 140×253)

Band Index	Pooled Output Size
0, 8, 16, 24	34×62
1, 9, 17, 25	27×49
2, 10, 18, 26	22×41
3, 11, 19, 27	19×35
4, 12, 20, 28	16×30
5, 13, 21, 29	14×27
6, 14, 22, 30	13×24
7, 15, 23, 31	11×22

Sample C1 Outputs



Figure 3: Sample C1 Outputs. These images represent the feature maps generated by the C1 layer after applying the max-pooling operation on the S1 outputs. Each map captures distinctive features with varying degrees of invariance to position and scale.

2.6 Patch Extraction from C1 Layer Outputs

In the training stage of the HMAX model, a large number of image patches are extracted from the C1 layer outputs. These patches are crucial for training the model and are used to recognize different objects. The extraction process is designed to randomly select patches of various sizes and positions from the C1 maps for each orientation (0, 45, 90, and 135).

The function `build_four_orientation_patch_pool` is responsible for generating a pool of patches. It takes as input the C1 layer outputs and produces patches of sizes ranging from 4×4 to 16×16 pixels. Each patch is composed of four channels, corresponding to the four orientations. The procedure for extracting patches is as follows:

1. The C1 maps are grouped by orientation, with each orientation containing eight different scale bands. Each scale band corresponds to a different receptive field size.
2. For each patch extraction, a random patch size (from the available sizes of 4, 8, 12, and 16 pixels) is chosen.
3. A valid scale band that has dimensions large enough to accommodate the selected patch size is randomly selected.
4. Random coordinates are chosen within the selected scale band, and the corresponding patch is extracted by slicing the C1 map.
5. The four extracted patches (one for each orientation) are concatenated to form a four-channel patch.

6. The patch, along with its metadata (e.g., size, scale band, and coordinates), is stored in the output list.

The extraction process is repeated until the desired number of patches (typically 1000) is reached. The extracted patches and metadata are then returned, ready for use in training the model.

This method ensures that the model learns to recognize features at various scales and orientations, thereby enhancing its ability to detect objects under different conditions.

Table 3: Distribution of Extracted Patches by Size

Patch Size	Number of Patches
4x4	248
8x8	248
12x12	235
16x16	269

2.7 S2 Layer: Algorithm and Implementation

The S2 layer in the HMAX model operates as a set of units that pool over the afferent C1 units from a local spatial neighborhood, across all four orientations. S2 units, functioning as radial basis function (RBF) units, enhance the invariance of the system to translation and scaling. These units compute responses based on the Euclidean distance between an input image patch and a stored prototype.

For an image patch X from the previous C1 layer at a specific scale S , the response r of a corresponding S2 unit is given by:

$$r = \exp(-\beta \|X - P_i\|^2)$$

where $\beta = 5$ controls the sharpness of the tuning, and P_i is one of the prototypes learned during the training phase.

At runtime, S2 maps are computed for all positions across the eight scale bands. These maps correspond to the responses of the system to each of the learned prototypes, which represent various object features extracted during the learning phase. The prototypes are distributed across different scale bands, and the model computes a map for each prototype in the corresponding scale band.

The core of the algorithm involves comparing the input image patches from the C1 layer to a set of prototypes using the Euclidean distance. Each scale band contains patches of different sizes, and the distance between the C1 features and the prototypes is computed at each position. The output of the S2 layer is a set of feature maps, where each map represents the best match between the input image and the learned prototypes at different positions and scales.

The key to the S2 layer’s functionality is the efficient matching process. Instead of computing the response for every possible position across all scale bands and orientations, the response is computed only for a

subset of locations. This reduces computational cost while maintaining the invariant properties of the system.

The S2 layer is implemented in such a way that it computes responses for each patch, combining across scales and orientations. The resulting feature maps from the S2 layer are then passed to the next stage of the model for further processing and classification. This process ensures that the model can robustly recognize objects, even when there are variations in size, position, and orientation.

In the implementation, patches are grouped by their sizes, and for each size group, a convolutional operation is performed using the corresponding patches as kernels. The convolutional filters are initialized with the patches and are not updated during training, thus ensuring the prototypical nature of the features. The results are then passed through an activation function (either Gaussian or ReLU), depending on the configuration, and the final feature maps are generated by concatenating the results from all scale bands.

2.7.1 Distance Calculation and Activation

In the S2 layer, the response of each unit is computed based on the distance between the input features from the C1 layer and the stored prototypes. The distance is primarily calculated using the **Euclidean distance** formula, which measures the straight-line distance between the input feature map X and the prototype P_i in the feature space. This distance calculation is done for every position in the input image and across all orientations.

The Euclidean distance is computed as:

$$\|X - P_i\|^2$$

After calculating the Euclidean distance, the result is processed by an activation function. If the activation type is set to Gaussian, the computed distance is transformed by a Gaussian kernel to model a similarity measure between the input and the prototypes. This transformation is given by:

$$r = \exp(-\beta \|X - P_i\|^2)$$

where β controls the sharpness of the tuning curve. In this implementation, β is set to 5, which determines the smoothness of the response. A larger value of β leads to a wider and smoother response, while a smaller value makes the response sharper and more sensitive to small variations in the distance. P_i represents the prototypes learned during training. The Gaussian function provides a smooth and localized response based on the distance between the input and the prototypes.

Alternatively, if the activation type is set to ReLU, the distance is processed by the ReLU function, followed by taking the square root of the result. This non-linear activation emphasizes the most relevant features and introduces sparsity in the output.

Alternatively, if the activation type is set to ReLU, the distance is processed by the ReLU function, fol-

lowed by taking the square root of the result. This non-linear activation emphasizes the most relevant features and introduces sparsity in the output.

Thus, the S2 layer incorporates both Euclidean distance and a transformation step (either Gaussian or ReLU) to compute how well an input feature map matches the learned prototypes. The Euclidean distance is primarily used.

2.8 S2 Layer

In this test, we compare an extracted patch from an airplane image with another airplane image. The comparison was done by calculating the Euclidean distance between the patch and the input image. The resulting

S2 map shows the distance between the two images, with the minimum value indicating a strong match between the features of the patch and the image.

As observed in the output, the ****minimum distance**** value is located in the region where the extracted patch and the input image share similar features, suggesting a high degree of similarity. This is consistent with the behavior of the Euclidean distance, where the smaller the distance, the greater the similarity between the two objects. The large area of minimum distance values in the S2 map suggests that the patch extracted from the airplane image aligns well with the input image, confirming the model's ability to detect and match similar features across different images.

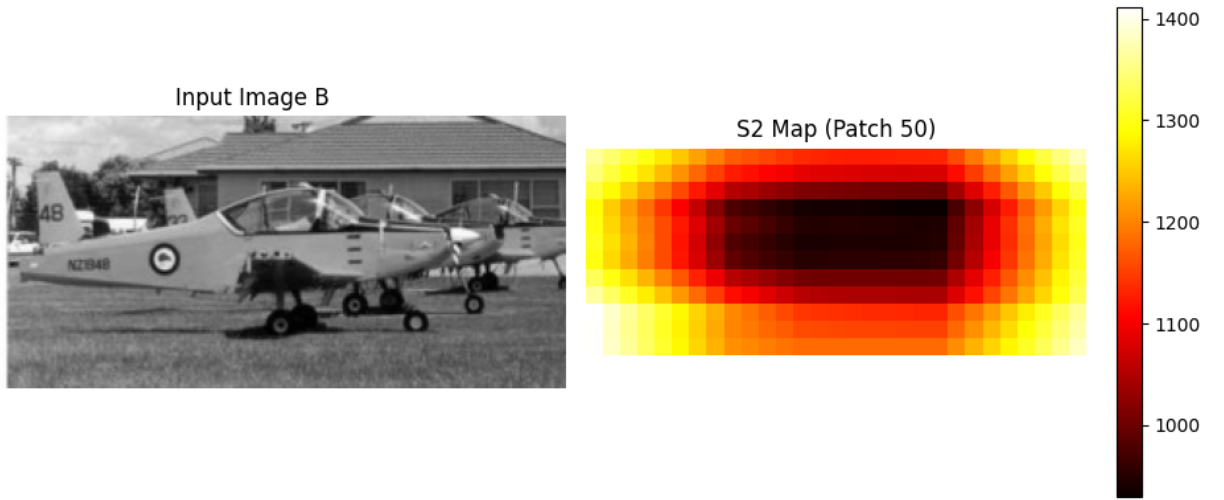


Figure 4: S2 Map Comparison: The patch from the airplane image is compared to another airplane image. The minimum Euclidean distance in the S2 map indicates the regions of similarity. The colorbar represents the distance values where lower values denote higher similarity.

2.9 C2 Layer

The C2 layer is responsible for further processing the features extracted from the S2 layer. It performs a pooling operation over the spatial dimensions of the feature maps, making the representation shift-invariant and scale-invariant. Specifically, the C2 layer applies either global min pooling or global max pooling, depending on the activation function used.

- If the activation is set to Euclidean, the minimum value is selected from each feature map across the spatial dimensions.
- If the activation is set to Gaussian or any other function, the maximum value is selected instead.

This operation produces a compact representation, making the C2 layer robust to variations in position and scale.

The output of the C2 layer is a tensor of shape $[B, N]$, where B is the batch size and N is the number of learned prototypes. This output is then forwarded to subsequent stages for comparison with learned prototypes.

For testing purposes, we extracted patches from an

airplane image and compared them against two different backgrounds: another airplane image and a random background. The goal was to observe how similar the patches from the airplane image were to those from a similar image (another airplane) versus a completely different background.

The results of this comparison showed that the patches from the airplane image were much closer to each other in terms of their minimal Euclidean distance when compared with another airplane. On the other hand, when compared to the background image, the minimal distances were significantly higher. This indicates that the patches from the airplane image are more similar to each other than to the background, highlighting the model's ability to recognize relevant features that closely match the target image.

Table 4: Patch Rankings

Rank	Patch ID	Score
Airplane-Airplane		
Rank 1	Patch 74	20.0776
Rank 2	Patch 34	23.1550
Rank 3	Patch 25	32.0969
Rank 4	Patch 77	32.2764
Rank 5	Patch 10	34.5034
Airplane-Background		
Rank 1	Patch 48	48.1934
Rank 2	Patch 12	57.6924
Rank 3	Patch 14	59.8021
Rank 4	Patch 76	61.0006
Rank 5	Patch 98	63.5681

2.10 Genetic Algorithm for Patch Selection

In the implementation of the Genetic Algorithm (GA) for patch selection, we aimed to optimize the set of patches used in the HMAX model to improve classification performance. Initially, the population consists of randomly initialized binary chromosomes, each representing a subset of patches from the patch pool. Each chromosome is evaluated by extracting features from the selected patches, training a classifier (SVM) on the extracted features, and measuring its accuracy on a test set. The fitness function is based on the classification accuracy, where a lower accuracy results in a higher fitness value (since we aim to minimize the fitness value).

The GA follows an iterative process across several generations. In each generation, the fittest individuals are selected based on their fitness score. These individuals undergo a crossover operation, where parts of two chromosomes are combined to form two offspring. Additionally, a mutation operation is applied with a small probability to introduce diversity in the population. This process of selection, crossover, and mutation continues for a predefined number of generations. The result of this process is the chromosome with the best selection of patches, which yields the highest classification performance. This method allows for efficient search and optimization of the patch set for better model accuracy.

2.11 Experiments and Results

In the first experiment, 1000 random patches were initially generated and used to evaluate the HMAX model. After generating these patches, the Genetic Algorithm (GA) was applied to select the most informative patches. The GA was used to select the top 200 patches based on their performance, and this selection process was considered as one run of the GA for testing. Additionally, for comparison, 200 random patches were selected from the original pool of 1000 patches, matching the number of patches selected by the GA. This setup produced a second model that was tested using the same process but with the randomly selected patches.

For the third model, another 1000 random patches

were generated independently, and the HMAX model was evaluated using this new set of patches. This experiment involved three different models: the first model used patches selected by the GA, the second model used random patches (with the same number of patches selected by the GA), and the third model used a completely random set of 1000 patches. These models were tested on the same test set to ensure a fair comparison, and the results were compared to evaluate the performance of the GA-based patch selection method against the random selection and full random patch models.

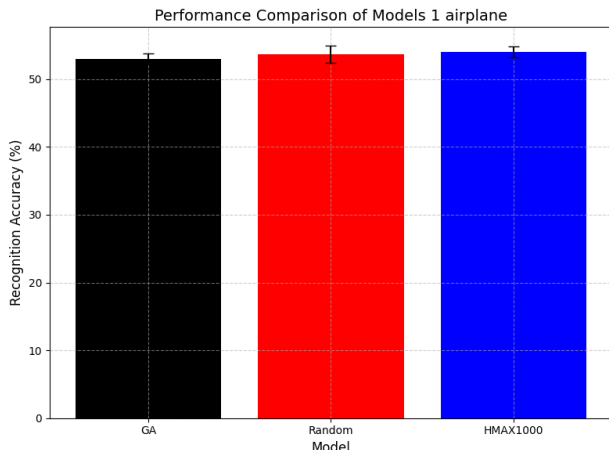
For the experiments, we utilized several datasets, including airplane, background, and face images, to evaluate the performance of the proposed model. All images were first converted to grayscale and resized to a height of 140 pixels, with the width adjusted proportionally to maintain the aspect ratio. This ensured that all images were consistent in size, with a height of 140 pixels across all samples.

The dataset was divided into positive and negative samples for training, validation, and testing. For the positive training samples, we used varying amounts of data with the following categories: 1, 3, 6, 15, 30, 40 images for each category. The negative training set consisted of 50 images. Additionally, for validation and evaluation, we used 40 positive samples (VAL_POS) and 40 negative samples (EVAL_NEG), with 50 positive (TEST_POS) and 50 negative (TEST_NEG) images used for testing. The total number of patches for the experiment was set to 1000, and a sigma value of 5.0 was used in the HMAX model.

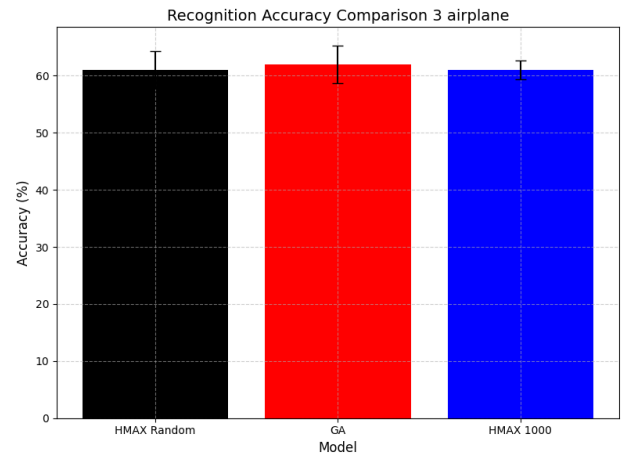
During the feature extraction process, images were passed through the S1 and C1 layers of the HMAX model, both of which utilized caching for efficiency. The Genetic Algorithm (GA) was configured with a population size of 5 and run for 5 generations, ensuring an efficient search for the optimal patch selection. All experiments were repeated across 3 random runs to account for variability, and the average performance along with the standard deviation were reported to ensure robustness in the results.

In Experiment 2, Figures presents a comparison between the HMAX model and the proposed model across two object classes : Motorbikes and Faces , using different numbers of features. In all cases, the proposed model outperforms the HMAX model. As previously mentioned, irrelevant patches, often coming from other object classes, can negatively impact the recognition performance and increase the complexity of the feature space. By eliminating these irrelevant patches, the quality of the features extracted in the C2 layer improves significantly. This reduction in feature space complexity leads to a noticeable increase in the model's overall performance, even when using a small number of patches.

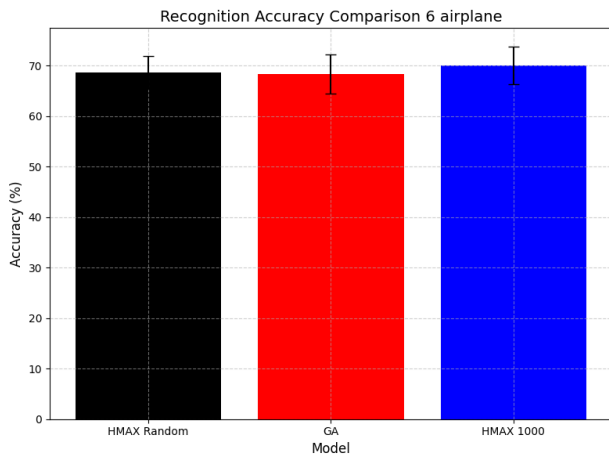
The code is available at the following Colab link: <https://colab.research.google.com/drive/1r-EjViPfsbIGZ3GcWQHMRDm5db9Bbz2a?usp=sharing>



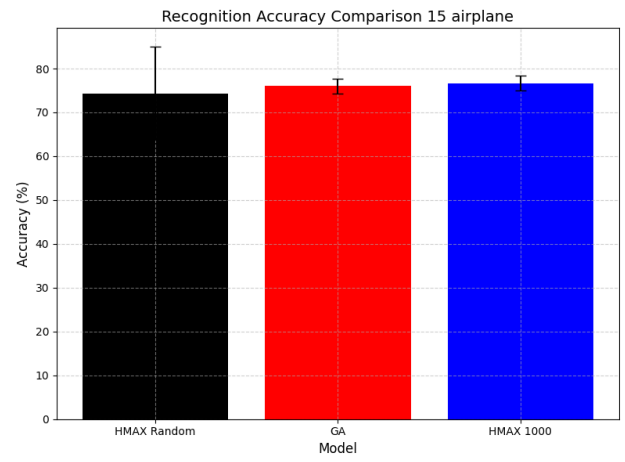
(a) 1 Training Sample



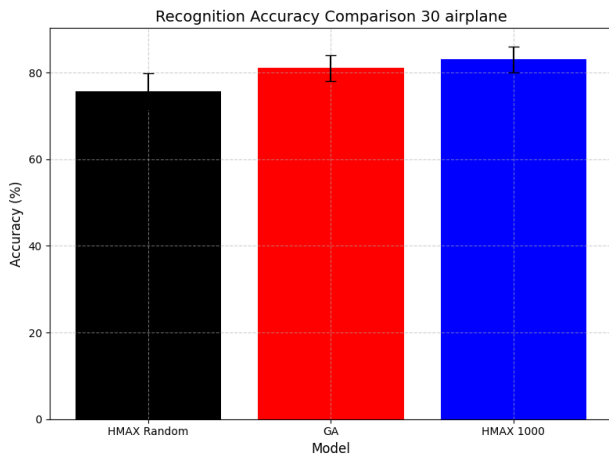
(b) 3 Training Samples



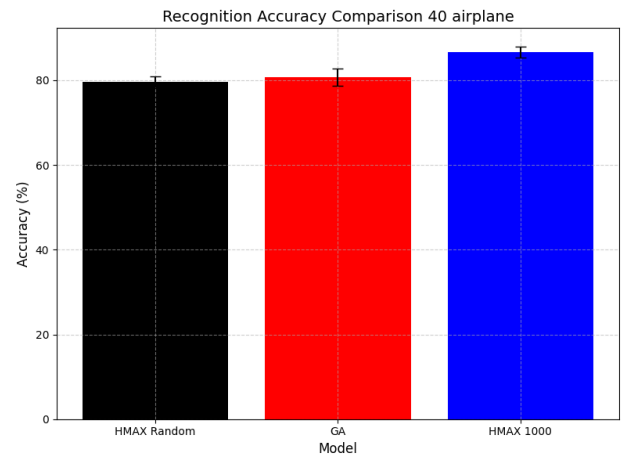
(c) 6 Training Samples



(d) 15 Training Samples

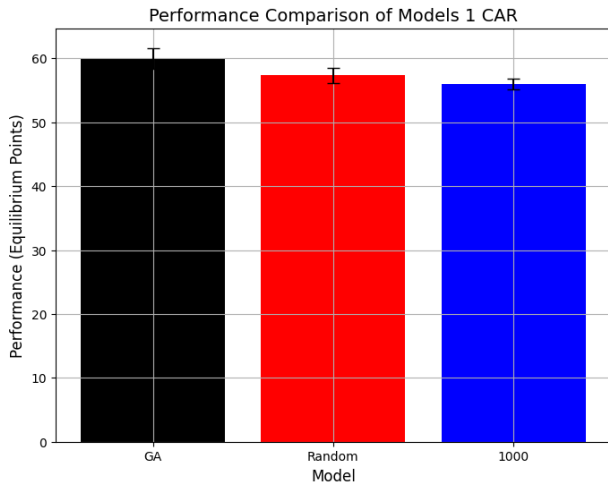


(e) 30 Training Samples

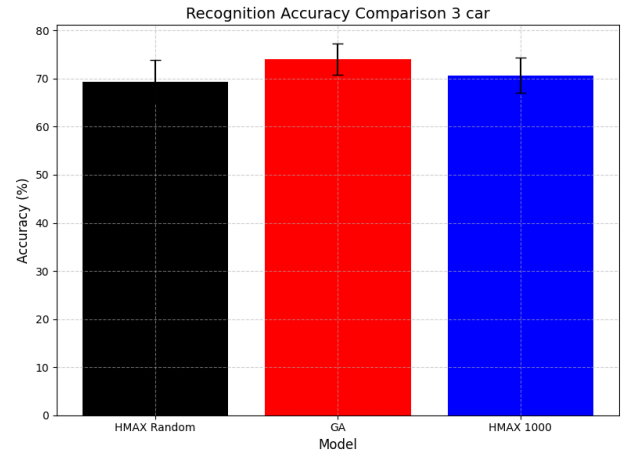


(f) 40 Training Samples

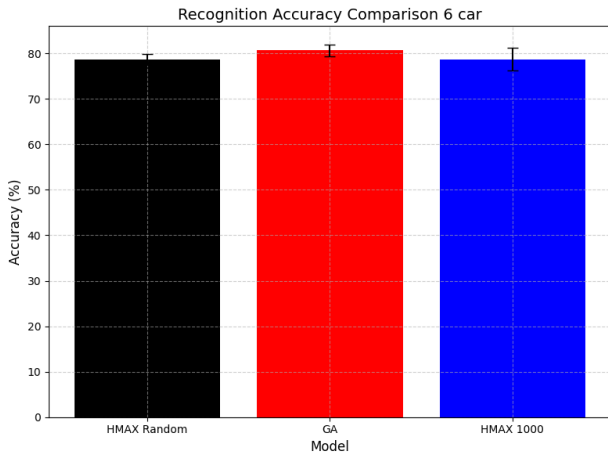
Figure 5: Recognition accuracy comparison for the **Airplane** category with different numbers of training samples.



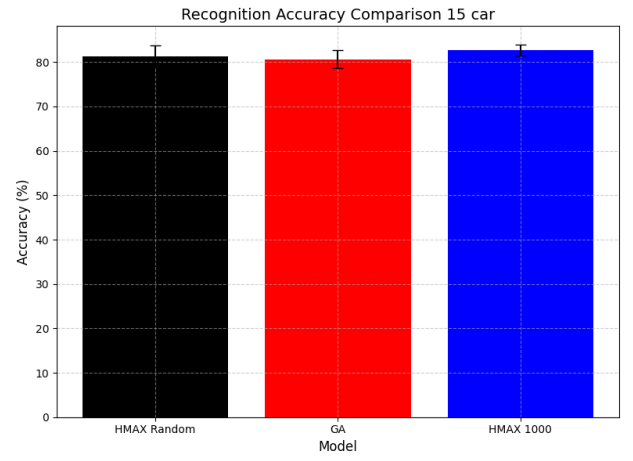
(a) 1 Training Sample



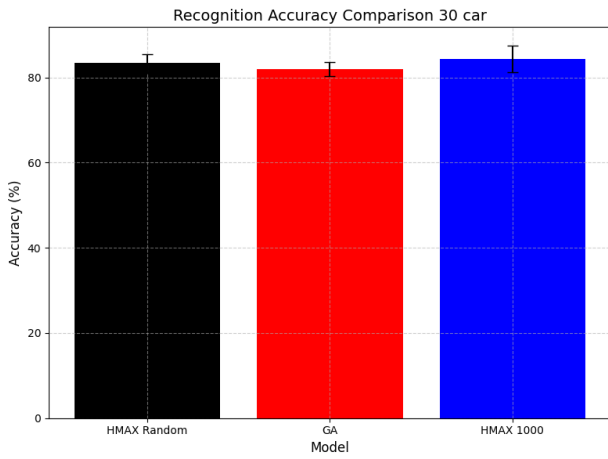
(b) 3 Training Samples



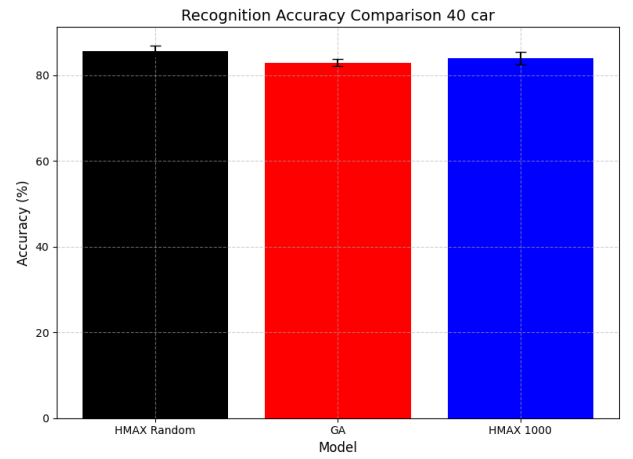
(c) 6 Training Samples



(d) 15 Training Samples

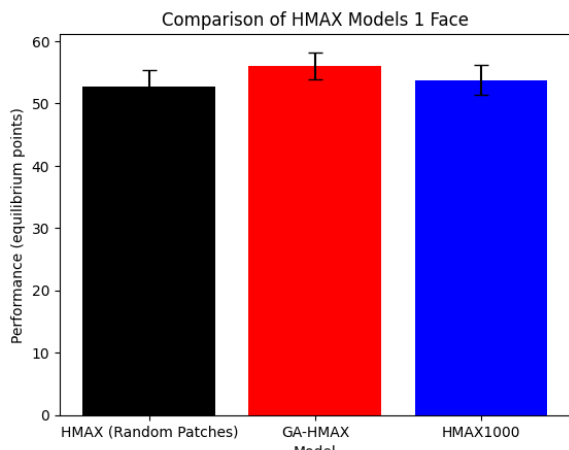


(e) 30 Training Samples

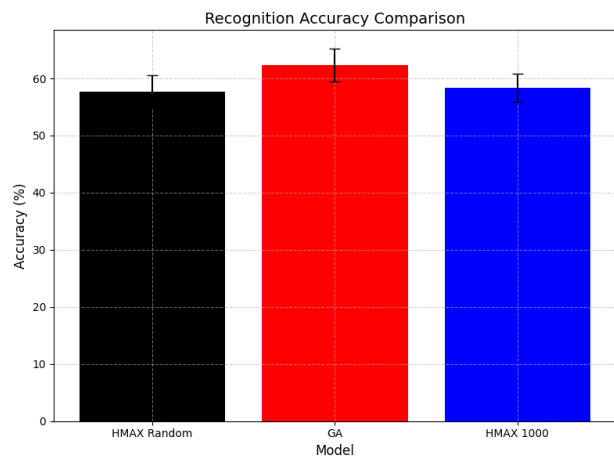


(f) 40 Training Samples

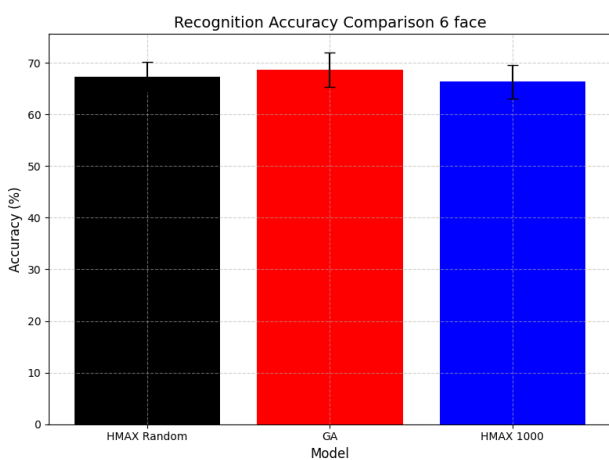
Figure 6: Recognition accuracy comparison for the **Car** category with different numbers of training samples.



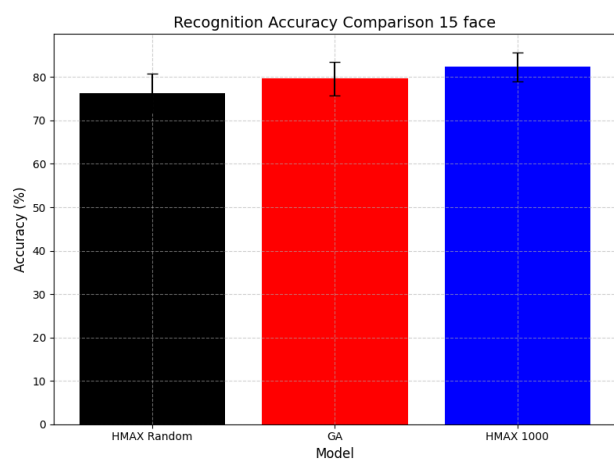
(a) 1 Training Sample



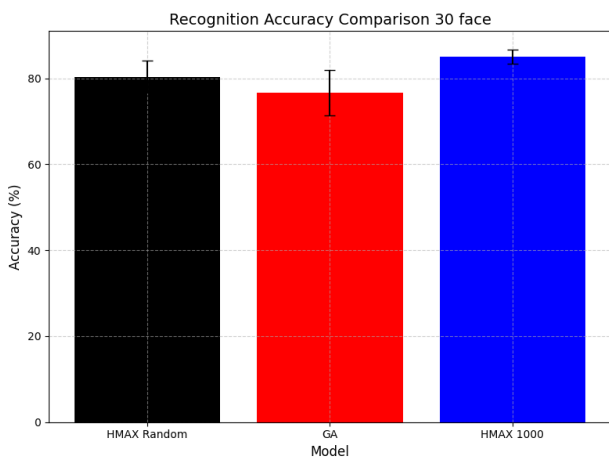
(b) 3 Training Samples



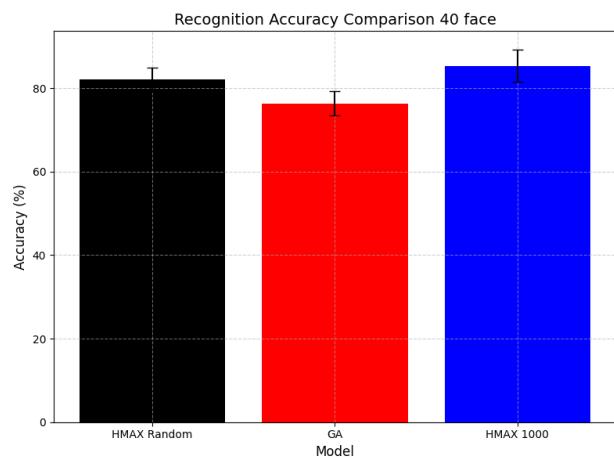
(c) 6 Training Samples



(d) 15 Training Samples



(e) 30 Training Samples



(f) 40 Training Samples

Figure 7: Recognition accuracy comparison for the **Face** category with different numbers of training samples.

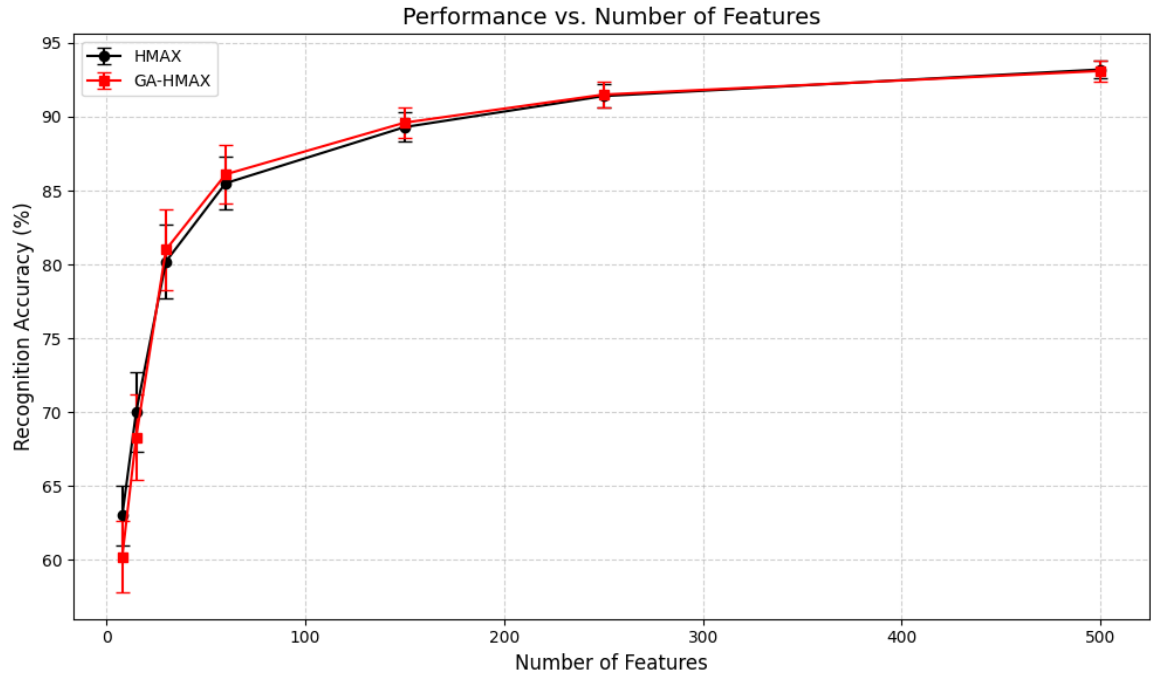


Figure 8: Performance vs. Number of Features for the Motorbike dataset. The GA-HMAX model (red) shows superior accuracy compared to the standard HMAX (black), especially with fewer features.

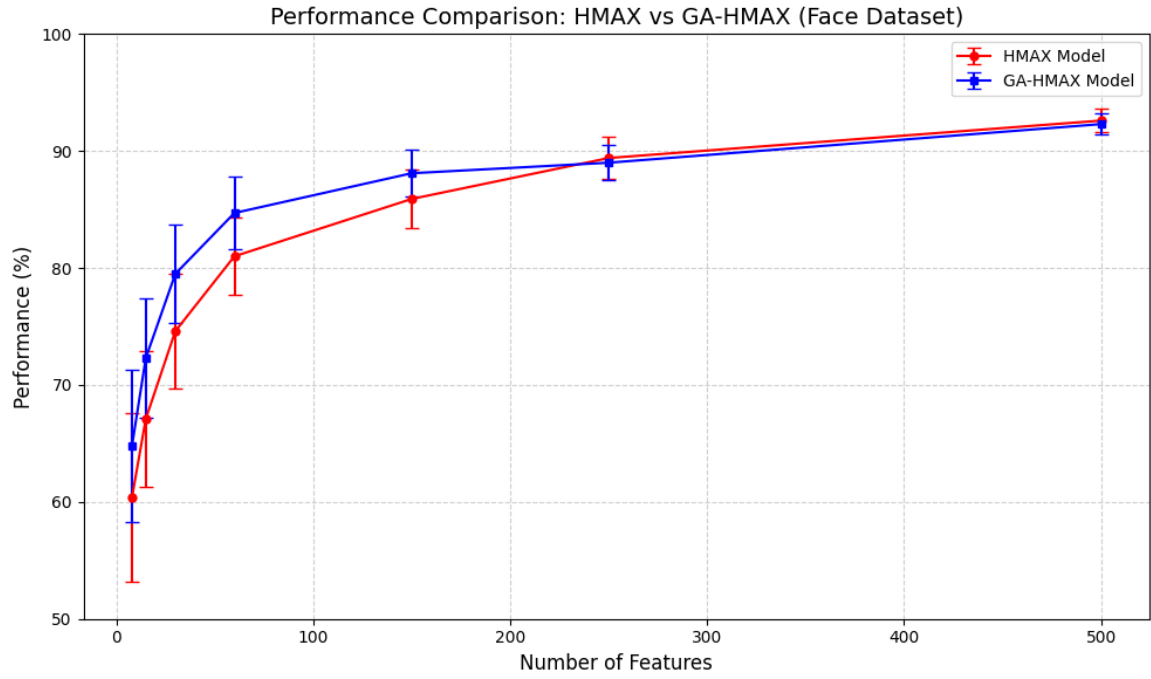


Figure 9: Performance comparison on the Face dataset. The GA-HMAX model (blue) achieves higher performance than the standard HMAX model (red) across different numbers of features, highlighting the benefit of optimized patch selection.